UNIVERSITY of NOTTINGHAM

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

HDL FOR PROGRAMMABLE DEVICES

MODULE CODE: EEEE4123

Project – Matrix Processing Core Design

1 INTRODUCTION

Matrix calculation is one of the fundamental mathematic calculations commonly used in advanced signal processing algorithms for a wide range of applications, such as satellite navigation systems, complex control systems and etc. In order to implement such advanced signal processing algorithms on an FPGA based embedded system, we need to use VHDL to design a matrix processing core for an AMD FPGA device.

2 PROJECT DESCRIPTION

In this project, you are asked to develop a synthesisable matrix processing core, which is able to do matrix multiplication as shown in Fig. 1 and then implement the core on the AMD FPGA device of xc7a15tcsg324-1. The matrix sizes are 8 x 16, 16 x 16 and 16 x 8 for input matrixes A, B and C, respectively, where each element is of 8-bit signed integer. Only behavioural simulation is required to verify your design, which must be synthesisable.

$$\mathbf{A}\begin{bmatrix} a11 & a12 & a13 & \dots & a116 \\ a21 & a22 & a23 & \dots & a216 \\ a31 & a32 & a33 & \dots & a316 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a81 & a82 & a83 & \dots & a816 \end{bmatrix} \times \mathbf{B}\begin{bmatrix} b11 & b12 & b13 & \dots & b116 \\ b21 & b22 & b23 & \dots & b216 \\ b31 & b32 & b33 & \dots & b316 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b161 & b162 & b163 & \dots & b1616 \end{bmatrix} \times \mathbf{C}\begin{bmatrix} c11 & c12 & c13 & \dots & c18 \\ c21 & c22 & c23 & \dots & c28 \\ c31 & c32 & c33 & \dots & c38 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c161 & c162 & c163 & \dots & c168 \end{bmatrix} = \mathbf{D}\begin{bmatrix} d11 & d12 & d13 & \dots & d18 \\ d21 & d22 & d23 & \dots & d28 \\ d31 & d32 & d33 & \dots & d38 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ d81 & d82 & d83 & \dots & d88 \end{bmatrix}$$

Fig. 1 Matrix Multiplication

The Matrix processing Core as shown in Fig. 2 mainly consists of two MACs (Multiply-Accumulate), three input buffers (InputBufferA, InputBufferB and InputBufferC), one temporary buffer (TempBuffer) and one output buffer (OutputBufferD). All these five buffers are simple dual-port block memories with part A for write and port B for read, which are created by using Block Memory Generator, which can be accessed from Vivado IP Catalog. Those three input buffers have the sizes of 64 x 8 bits, 128 x 8 bits and 64 x 8 bits, which are used to receive the data for matrix A, matrix B and matrix C, respectively. The temporary buffer stores the results of the 1st matrix

multiplication (A x B). The output buffer has a size of 64 x 32 bits, which is for the results of the 2nd matrix calculation. The data organisation of these four buffers is shown in Fig. 3.

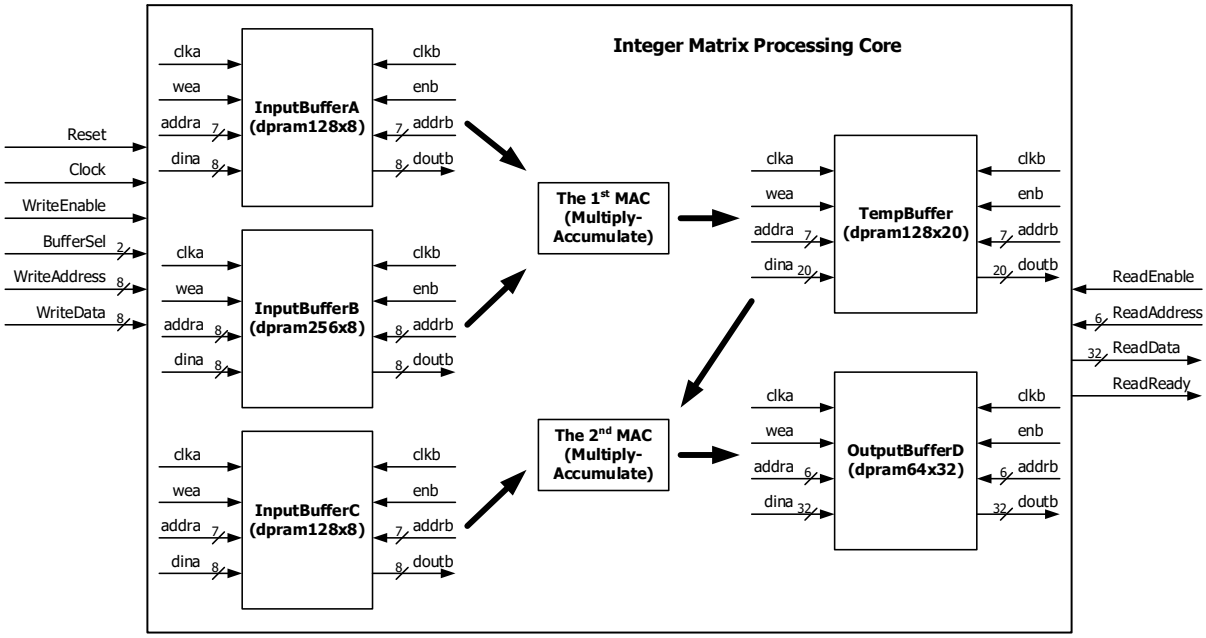

Fig. 2 Matrix Processing Core

InputBufferA (dpram128x8)		InputBufferB (dpram256x8)		InputBufferC (dpram128x8)		TempBuffer (dpram128x20)		OutputBufferD (dpram64x32)	
0	a11	0	b11	0	c11	0	t11	0	d11
1	a12	1	b12	1	c12	1	t12	1	d12
2	a13	2	b13	2	c13	2	t13	2	d13
⋮		⋮		⋮		⋮		⋮	
15	a116	15	b116	8	c18	15	t116	7	d18
16	a21	16	b21	9	c21	16	t21	8	d21
17	a22	17	b22	10	c22	17	t22	9	d22
18	a23	18	b23	11	c23	18	t23	10	d23
⋮		⋮		⋮		⋮		⋮	
31	a216	31	b216	15	c28	31	t216	15	d28
32	a31	32	b31	16	c31	32	t31	16	d31
33	a32	33	b32	17	c32	33	t32	17	d32
34	a33	34	b33	18	c33	34	t33	18	d33
⋮		⋮		⋮		⋮		⋮	
47	a316	47	b316	23	c38	47	t316	23	d38
⋮		⋮		⋮		⋮		⋮	
112	a81	240	b161	120	c161	112	t81	56	d81
113	a82	251	b162	121	c162	113	t82	57	d82
114	a83	252	b163	122	c163	114	t83	58	d83
⋮		⋮		⋮		⋮		⋮	
127	a816	255	b1616	127	c168	127	t816	63	d88

Fig. 3 Buffer Data Organisation

Write/Read timing requirements are shown in Fig. 4 and Fig. 5, respectively. In order to write a data to a particular address of the dual-port RAM, the valid data and address to be written must be

presented at the signals, ‘dina’ and ‘addra’ at the rising edge of the clock with ‘1’ for signal ‘wea’ at the same rising edge. From the read operation, the address to be read must be presented at signal, ‘addrb’ at the rising edge of the clock with ‘1’ for signal ‘enb’ at the same rising edge. The read data will be available at signal ‘doutb’ just after the rising edge.

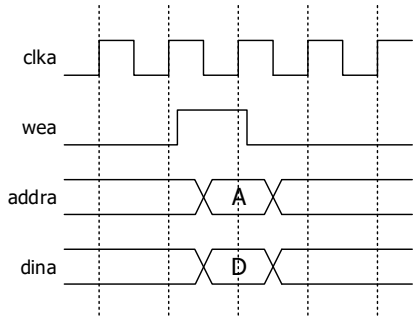

Fig. 4 Timing for Write Operation to the Dual-Port RAM

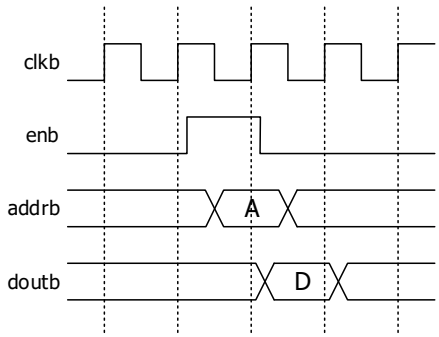

Fig. 5 Timing for Read Operation to the Dual-Port RAM

3 DESIGN TASKS

Design a synthesisable matrix processing core and its testbench.

The specific design requirements are as follows:

- To facilitate the assessment of your matrix processing core design, you must use the following entity declaration for the top level design.

```
-- Required entity declaration
entity IntMatProCore is
  port(
    Reset, Clock, WriteEnable:in std_logic;
    -- BufferSel: "00" for input buffer A, "01" for input buffer B, "10"
for input buffer C
    BufferSel:    in std_logic_vector (1 downto 0);
    WriteAddress: in std_logic_vector (7 downto 0);
    WriteData:    in std_logic_vector (7 downto 0);

    ReadAddress:  in std_logic_vector (5 downto 0);
    ReadEnable:   in std_logic;
    ReadData:     out std_logic_vector (31 downto 0);
```

```
      DataReady:    out std_logic
   );
end IntMatProCore;
```

- In the testbench, you must use VHDL file I/O to read the data for three input matrixes from three input text files (inputA.txt, inputB.txt and inputC.txt) and then feed them to the design. Once the matrix calculation is completed, you must read out the results from the output buffer, compared with the output data (outputD_matlab.txt) from your Matlab model and write output data from your VHDL simulation and the comparison results into an output text file (outputD.txt). Please refer to the reference design provided.

- The testbench must produce all the input signals and the simulation waveforms must show how the data of the matrixes A/B/C are written into the input buffers and how the matrix calculation results are read out from the output buffer.

- The description of interface signals:

Signal	**Description**
Reset	Synchronous reset, active high, to reset the matrix multiplier core. Whenever issued, it lasts 10 cycles.
Clock	The global clock signal for the matrix multiplier core.
WriteEnable	Active high, to enable write the data of matrixes A and B into the input buffers, InputBufferA and InputBufferB, respectively. 'BufferSel' will design which input buffer will be written.
BufferSel	"00" for input buffer A, "01" for input buffer B, "10" for input buffer C.
WriteAddress	Write address signal for InputBufferA/B/C.
WriteData	Data signal for matrixes A/B/C. 'BufferSel' will decide which data should be presented at 'WriteData'.
ReadAddress	Read address signal of the outputbuffer, OutputBufferD.
ReadEnable	Active high, to enable read operation for the outputbuffer.
ReadData	Whenever the ReadEnable signal is high at the rising edge of the Clock, the data in the output buffer pointed by ReadAddress at the same rising edge will appear at ReadData immediately after the same rising edge.
DataReady	Active high, to indicate that the matrix calculation results have been written into the output buffer and are ready for reading out from part B of the output buffer. DataReady must last one cycle.

4 REFERENCE DESIGN

To get your VHDL project design started easily, a synthesisable matrix multiplier core design is provided as a reference design (Fig. 6/Fig. 7), which is able to do matric multiplication for matrixes

with the size of 4 x 4 and an 8-bit signed integer for each element of the input matrixes (A and B). The data buffers will be organised as Fig. 8.

$$\overset{\mathbf{A}}{\begin{bmatrix} a11 & a12 & a13 & a14 \\ a21 & a22 & a23 & a24 \\ a31 & a32 & a33 & a34 \\ a41 & a42 & a43 & a44 \end{bmatrix}} \times \overset{\mathbf{B}}{\begin{bmatrix} b11 & b12 & b13 & b14 \\ b21 & b22 & b23 & b24 \\ b31 & b32 & b33 & b34 \\ b41 & b42 & b43 & b44 \end{bmatrix}} = \overset{\mathbf{C}}{\begin{bmatrix} c11 & c12 & c13 & c14 \\ c21 & c22 & c23 & c24 \\ c31 & c32 & c33 & c34 \\ c41 & c42 & c43 & c44 \end{bmatrix}}$$

Fig. 6 Matrix Multiplication

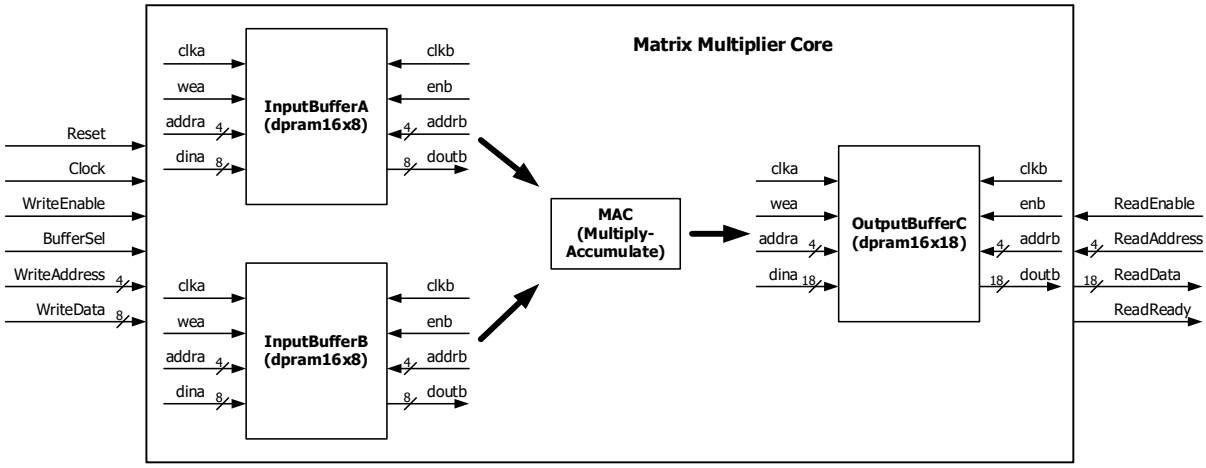

Fig. 7 Matrix Multiplier Core

	InputBufferA (dpram16x8)		InputBufferB (dpram16x8)		OutputBufferC (dpram16x18)
0	a11	0	b11	0	c11
1	a12	1	b12	1	c12
2	a13	2	b13	2	c13
3	a14	3	b14	3	c14
4	a21	4	b21	4	c21
5	a22	5	b22	5	c22
6	a23	6	b23	6	c23
7	a24	7	b24	7	c24
8	a31	8	b31	8	c31
9	a32	9	b32	9	c32
10	a33	10	b33	10	c33
11	a34	11	b34	11	c34
12	a41	12	b41	12	c41
13	a42	13	b42	13	c42
14	a43	14	b43	14	c43
15	a44	15	b44	15	c44

Fig. 8 Memory buffer organisation for the matrix multiplier core

5 DESIGN TOOLS

The project design tools must be Vivado 2018.3 with either the built-in Vivado simulator or ModelSim 10.6c

6 PROJECT SUBMISSION

The VHDL project must be submitted on Moodle by **3:00pm, Tuesday, 5th May 2026**. The submission must have a single zip file, which includes the following:

- A word file – 10-page project report
- VHDL source codes
- Matlab codes
- inputA.txt, inputB.txt, inputC.txt – Input test data generated from your Matlab model
- outputD_matlab.txt – Output test data generated from your Matlab model
- outputD.txt – Output data produced from your VHDL testbench and the comparison result between the output data from your Matlab model and the output data from your VHDL simulation
- Any other relevant files
- **An entire Vivado project folder for your VHDL project design** (If it is too large to be submitted, then you need to re-create the Vivado project with the final VHDL codes and test data files)

NOTE: You should not put anything identifying yourself (e.g. your name or your student ID) in your submission.

The marking scheme of the VHDL project is as follows:

- Design, Implementation and Verification (60 marks):
  1. Matlab model (5 marks)
     - Signed integer data
     - Able to automatically generate all the test files
  2. VHDL code compilation (5 marks)
     - No compilation errors
  3. Synthesis (15 marks)
     - No synthesis errors, such as multi-drive errors
     - No critical warnings
     - Allow non-critical warnings, which cannot be ignored, but 5 marks' penalty per non-critical warning.
  4. Behavioural simulation (5 marks)
     - Behavioural simulation can be launched without any errors
  5. Correct data (10 marks)
     - The result data from VHDL simulation must match that generated from your Matlab model
  6. Design efficiency (10 marks)
     - It is assessed, in terms of the number of the clock cycles from the rising edge of WriteEnable to the rising edge of DataReady
  7. Coding quality and readability (10 marks)

    - Must use one state machine with two processes, one of which is a clocked process and the other is purely combinational process
    - Concise code comments
- 10-page report (40 marks):
  1. Description of project design (10 marks)
     - A state transition diagram must be included
  2. Simulation and discussion (10 marks)
     - Including simulation waveforms to show those key internal signals to justify your verification process
  3. Conclusions (6 marks)
  4. Further work (4 marks)
  5. Report quality (10 marks)
     - Clarity
     - Appropriate format and structure
     - Length

NOTE: A penalty of 5% per working day will be imposed for the late submission.